Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

Group Detail

Name: SRINITHA M

College: KPR College Of Arts Science And Research

Contact: 23bcs059@kprcas.ac.in

Name: BHADRA U

College: KPR College Of Arts Science And Research

Contact: 23bcs012@kprcas.ac.in

Name: KAVIYA S

College: KPR College Of Arts Science And Research

Contact: 23bcs031@kprcas.ac.in

Name: NIRMALKUMAR S S

College: KPR College Of Arts Science And Research

Contact: 23bcs040@kprcas.ac.in

Name: KAVIYARASU M

College: KPR College Of Arts Science And Research

Contact: 23bcs033@kprcas.ac.in

Project Title: Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

Overview:

Context-aware access control is a modern security approach that determines whether a user should be granted access to a system, application, or data based on various contextual factors rather than relying only on basic authentication such as a username and password. In traditional access control systems, once a user's identity is verified, access is usually granted according to predefined roles or permissions. However, this approach may not be sufficient in today's dynamic and distributed digital environments where users access resources from different devices, networks, and locations. Context-aware access control enhances security by continuously evaluating multiple conditions related to the access request before allowing or restricting access.

This model typically analyzes several key factors including the user's identity, the security status of the device being used (device posture), the geographic or network location from which the request is made, and the risk level associated with the application or resource being accessed. For example, a verified employee using a company-managed laptop from a trusted office network may be granted full access to sensitive applications. In contrast, the same employee attempting to access the same system from an unknown device or public network may be required to complete additional authentication steps or may only receive limited access. This dynamic evaluation helps organizations ensure that access decisions are not only based on who the user is, but also on the context in which the access request occurs.

Context-aware access control is widely used in modern enterprise security frameworks such as Zero Trust Architecture, where the principle of "never trust, always verify" is applied. It plays a critical role in protecting sensitive data, preventing unauthorized access, and reducing security risks in environments where employees, contractors, and partners frequently access resources remotely or through cloud-based applications. By continuously monitoring contextual signals and adjusting access permissions accordingly, organizations can maintain a balance between strong security and user convenience while safeguarding critical systems and information.

Abstract

In modern digital environments, organizations face increasing cybersecurity threats due to remote access, cloud computing, and distributed networks. Traditional access control mechanisms that rely only on usernames and passwords are no longer sufficient to protect sensitive systems and data. This project focuses on designing and applying context-aware access control mechanisms based on multiple security factors such as user identity, device posture, location, and application risk level. The goal of the project is to improve security by dynamically evaluating the context of each access request before granting permission to users.

In the first stage of the project, the concept of context-aware access control was studied in detail, including its role in modern security frameworks such as the Zero Trust Architecture. Various access scenarios were analyzed to understand how organizations can control access based on contextual information. The second stage of the project focused on vulnerability assessment using the tool Nessus. The tool was used to scan a target web application and identify potential security weaknesses such as open ports, detected services, and system configuration details. The results demonstrated how vulnerability scanning helps organizations identify and manage security risks before they can be exploited by attackers.

In the final stage, the project analyzed important components of context-aware security including user identity verification, device security posture validation, and location-based access control policies. These mechanisms help ensure that only authorized and trusted users with secure devices can access sensitive applications and resources. The study highlights the importance of adaptive security systems that continuously evaluate contextual factors to prevent unauthorized access and reduce cybersecurity risks.

Use Case Scenarios:

1. Remote Employee Accessing Company Email

Scenario:
An employee tries to access company email from home.

Context Evaluation

- User identity: Verified through Multi-Factor Authentication (MFA).
- Device posture: Personal laptop without latest security patches.
- Location: Home network (trusted country).
- Application risk: Medium (email contains internal communication).

Access Control Decision

- Allow access only through web interface.
- Block download of attachments.
- Require step-up authentication.

Result:
Employee can read email but cannot download sensitive files.

2. Finance Manager Accessing Payroll System

Scenario:
A finance manager logs into the payroll system.

Context Evaluation

- User identity: Finance Manager role.
- Device posture: Corporate laptop with antivirus and disk encryption.
- Location: Inside corporate office network.
- Application risk: High (salary and banking data).

Access Control Decision

- Grant full access to payroll system.

- Allow editing and exporting data.

Result:
Trusted environment allows normal operations.

3. Suspicious Login from Unusual Location

Scenario:
An employee attempts login from another country.

Context Evaluation

- User identity: Valid credentials.
- Device posture: Unknown device.
- Location: Foreign country not normally used by employee.
- Application risk: High (internal HR system).

Access Control Decision

- Block access temporarily.
- Trigger security alert to SOC team.
- Ask user to verify identity.

Result:
Potential account compromise is prevented.

4. Contractor Accessing Project Management Tool

Scenario:
A third-party contractor logs in to manage tasks.

Context Evaluation

- User identity: Contractor account.
- Device posture: Personal device.
- Location: Remote.
- Application risk: Low (task tracking system).

Access Control Decision

- Allow limited access only to assigned projects.
- Prevent access to company internal documents.

Result:
Contractor can collaborate without exposing sensitive data.

5. Mobile Access to Sensitive Application

Scenario:
A sales employee opens CRM from a mobile phone.

Context Evaluation

- User identity: Verified employee.
- Device posture: Mobile phone without company MDM policy.
- Location: Public Wi-Fi in airport.
- Application risk: High (customer data).

Access Control Decision

- Allow read-only access.
- Block data export and downloads.

Result:
User can check customer details but cannot extract data.

STAGE-1

Top 10 OWASP:

No.	Vulnerability Category	Vulnerability Name	CWE ID
1	Broken Access Control	Improper Authorization / Privilege Escalation	CWE-284
2	Cryptographic Failures	Sensitive Data Exposure due to Weak Encryption	CWE-522
3	Injection	SQL/Command/LDAP Injection	CWE-89
4	Insecure Design	Missing Security Controls in Architecture	CWE-657
5	Security Misconfiguration	Improper Security Configuration	CWE-16
6	Vulnerable and Outdated Components	Use of Vulnerable Libraries/Dependencies	CWE-1104
7	Identification and Authentication Failures	Weak Authentication / Session Management	CWE-287
8	Software and Data Integrity Failures	Unsigned or Unverified Software Updates	CWE-494
9	Security Logging and Monitoring Failures	Insufficient Logging and Alerting	CWE-778
10	Server-Side Request Forgery (SSRF)	Improper URL Validation	CWE-918
11	Improper Network Segmentation	Lack of Isolation Between Services	CWE-653
12	Insufficient Transport Layer Protection	Weak TLS Configuration	CWE-319
13	Improper Identity Management	Insecure Identity Federation / SSO Issues	CWE-306
14	Cloud Misconfiguration	Misconfigured Cloud Storage or Access Policies	CWE-284
15	Insufficient Device Trust Validation	Unverified Endpoint or Device Posture	CWE-295

Reports:

Vulnerability Report 1

Vulnerability name: Cross Site Scripting (Reflected)
CWE: 79
OWASP category: A03:2021 - Injection

Description:
Reflected Cross-Site Scripting (XSS) occurs when a web application includes untrusted user input in its response without properly validating or escaping it. The malicious script is reflected off the web server and executed in the victim's browser. Attackers exploit this vulnerability by embedding malicious scripts in URLs or form inputs.

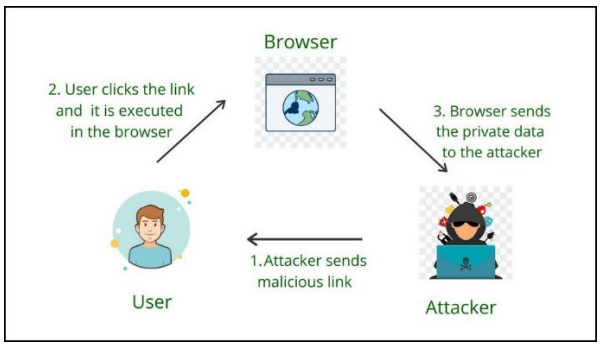
Business Impact:
Attackers can steal session cookies, redirect users to malicious websites, capture sensitive information, or perform actions on behalf of the victim. If the attack targets privileged users such as administrators,

Vulnerability path:
https://portswigger-labs.net/

Vulnerability parameter:
/search?query=%3Cscript%3Ealert('hacked')%3C%2Fscript%3E

Steps to Reproduce:
➢ Access the vulnerable search page.
➢ In the search input field, enter the payload.
➢ The payload used is <script>alert('hacked')</script>.
➢ Submit the search request.
➢ The application reflects the payload in the response page and executes the script in the browser.

Recommendation:
• Validate and sanitize all user input before processing it.
• Implement proper output encoding when displaying user-controlled data.
• Use Content Security Policy (CSP) to prevent execution of unauthorized scripts.
• Implement web application firewalls to detect malicious input patterns.



Vulnerability Report 2

Vulnerability name: SQL Injection
CWE: 89
OWASP category: A03:2021 - Injection

Description:
SQL Injection occurs when an application fails to properly validate user input before including it in SQL queries. Attackers can manipulate database queries by inserting malicious SQL code into input fields. This allows attackers to access, modify, or delete data stored in the database.

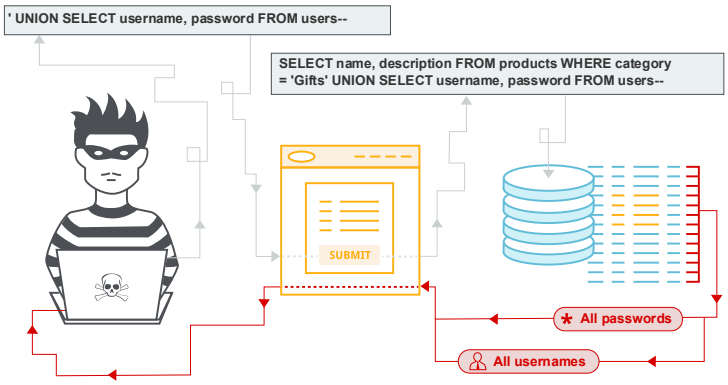
Business Impact:
Successful SQL injection attacks may expose sensitive customer data, login credentials, financial information, or confidential business records. Attackers can bypass authentication, extract entire databases, or even gain administrative privileges within the application.

Vulnerability path:
https://portswigger-labs.net/login

Vulnerability parameter:
/login?username=admin'--&password=test

Steps to Reproduce:
➢ Access the login page.
➢ Enter the username payload admin'--.
➢ Enter any password value.
➢ Submit the login form.
➢ The SQL query becomes manipulated and authentication is bypassed.

Recommendation:
• Use parameterized queries or prepared statements for database interactions.
• Implement strict input validation and sanitization.
• Use stored procedures where possible.
• Apply least privilege access controls to database accounts.



Vulnerability Report 3

Vulnerability name: Broken Access Control
CWE: 284
OWASP category: A01:2021 - Broken Access Control

Description:
Broken access control occurs when an application fails to properly enforce user authorization policies. This allows attackers to access restricted resources or perform actions beyond their privileges by manipulating requests or parameters.

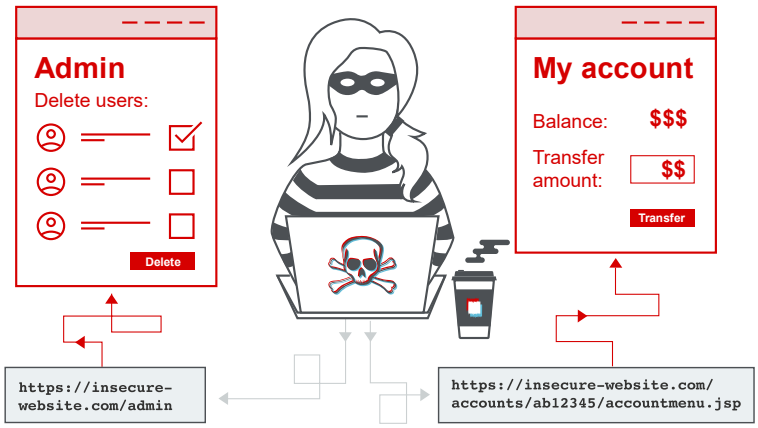
Business Impact:
Attackers can access sensitive data, modify or delete records, or perform administrative operations without authorization. This can lead to data breaches, financial loss, and reputational damage for the organization.

Vulnerability path:
https://portswigger-labs.net/admin

Vulnerability parameter:
/admin/delete?username=carlos

Steps to Reproduce:
➢ Log in as a normal user.
➢ Intercept the request using a proxy tool.
➢ Modify the URL to access /admin/delete?username=carlos.
➢ Forward the request to the server.
➢ The application performs the action without verifying user permissions.

Recommendation:
• Implement proper role-based access control (RBAC).
• Verify authorization checks on both client and server sides.
• Restrict access to sensitive endpoints based on user roles.
• Log and monitor unauthorized access attempts.



Vulnerability Report 4

Vulnerability name: Cross Site Request Forgery (CSRF)
CWE: 352
OWASP category: A01:2021 - Broken Access Control

Description:
Cross-Site Request Forgery occurs when an attacker tricks an authenticated user into performing unwanted actions on a web application. The application fails to verify whether the request was intentionally sent by the user, allowing attackers to exploit authenticated sessions.

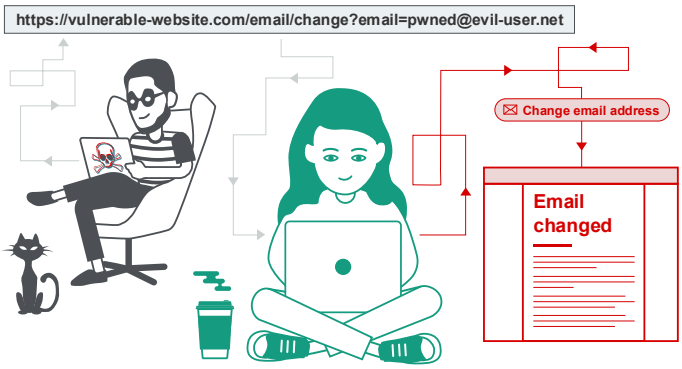
Business Impact:
Attackers can perform unauthorized actions such as changing user passwords, transferring funds, or modifying account information. If successful against privileged users, the attacker may compromise critical system functionality.

Vulnerability path:
https://portswigger-labs.net/account

Vulnerability parameter:
/account/change-email

Steps to Reproduce:
➢ Log in to the application.
➢ Navigate to the email change functionality.
➢ Capture the request and observe the absence of a CSRF token.
➢ Create a malicious HTML form that submits the same request automatically.
➢ When a logged-in user visits the malicious page, the email address is changed without their consent.

Recommendation:
• Implement CSRF tokens in all state-changing requests.
• Verify the Origin and Referer headers.
• Use SameSite cookie attributes.
• Require re-authentication for sensitive operations.



Vulnerability Report 5

Vulnerability name: Server-Side Request Forgery (SSRF)
CWE: 918
OWASP category: A10:2021 - Server-Side Request Forgery

Description:
Server-Side Request Forgery occurs when a web application fetches remote resources based on user-supplied input without proper validation. Attackers can manipulate the request to access internal systems or services that are not normally exposed to the public.

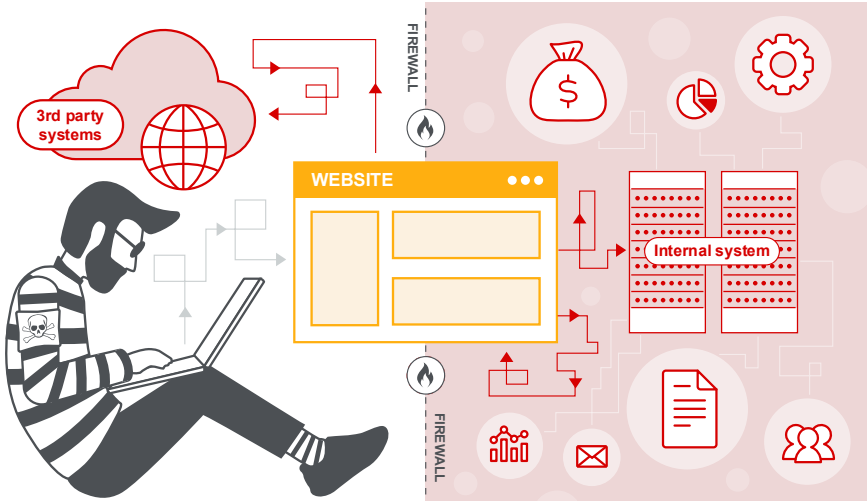
Business Impact:
SSRF vulnerabilities may allow attackers to access internal services, cloud metadata endpoints, or administrative interfaces. This can result in sensitive data exposure, infrastructure compromise, or privilege escalation.

Vulnerability path:
https://portswigger-labs.net/product/stock

Vulnerability parameter:
/stockApi?stockApi=http://localhost/admin

Steps to Reproduce:
➢ Access the product stock checking feature.
➢ Intercept the request using a proxy tool.
➢ Modify the stockApi parameter to point to an internal address.
➢ Send the request to the server.
➢ The server processes the request and returns internal data.

Recommendation:
• Implement strict allowlists for external resource requests.
• Block requests to internal IP addresses and localhost.
• Disable unnecessary URL fetching features.
• Monitor outgoing server requests for suspicious patterns.



Vulnerability Report 6

Vulnerability name: Use of Vulnerable Libraries/Dependencies
CWE: 1104
OWASP category: A06:2021 – Vulnerable and Outdated Components

Description:
This vulnerability occurs when applications use outdated or unsupported libraries and software components that contain known security vulnerabilities. Attackers can exploit these vulnerabilities to compromise the application or underlying system. Since these vulnerabilities are publicly documented, attackers can easily use automated tools to identify and exploit them.

Business Impact:
Outdated components may allow attackers to execute malicious code, gain unauthorized access, or compromise system integrity. If a widely used library contains vulnerabilities, multiple applications within the organization may be affected, increasing the risk of large-scale security incidents.

Vulnerability path:
http://localhost/juice-shop

Vulnerability parameter:
Outdated dependency version in application package files.

Steps to Reproduce:
➢ Run a dependency scanning tool on the application.
➢ Identify outdated or vulnerable packages.
➢ Observe security alerts for known CVEs related to the components.

Screenshot Process:

```python
from flask import Flask, request
import secure_input_lib  # Secure updated library

app = Flask(__name__)

@app.route('/process', methods=['POST'])
def process_input():
    data = request.json.get('input')
    if not isinstance(data, str):
        return {"error": "Invalid input"}, 400  # Extra validation
    try:
        result = secure_input_lib.process(data)  # Secure call
        return {"result": result}, 200
    except Exception as e:
        return {"error": str(e)}, 500

if __name__ == '__main__':
    app.run(port=3000)
```

Recommendation:
• Regularly update all software dependencies.
• Monitor vulnerability databases for known component vulnerabilities.
• Use automated dependency scanning tools during development.

Vulnerability Report 7

Vulnerability name: Weak Authentication / Session Management
CWE: 287
OWASP category: A07:2021 – Identification and Authentication Failures

Description:
Weak authentication vulnerabilities occur when applications do not properly enforce strong authentication mechanisms. This may include weak passwords, lack of account lockout policies, or insecure session management practices.

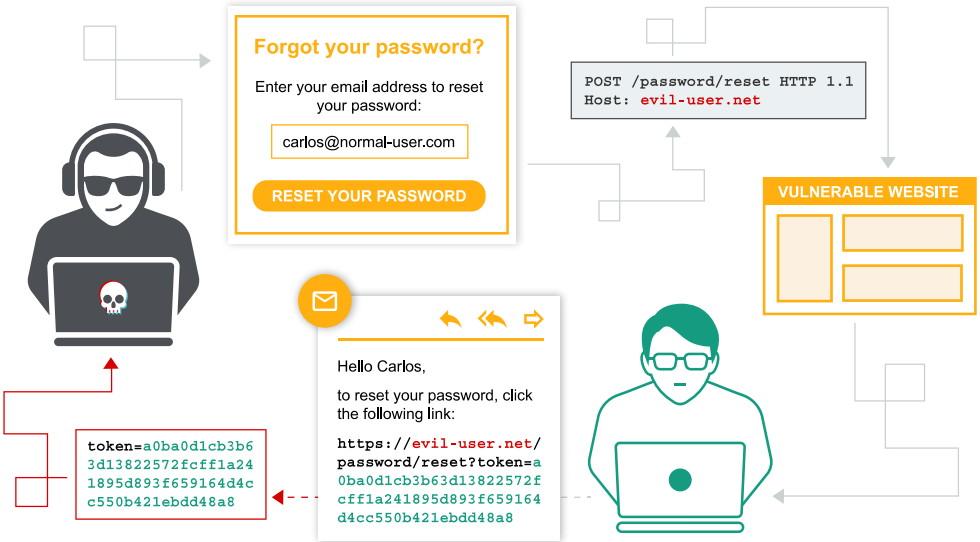
Business Impact:
Attackers may exploit weak authentication controls to gain unauthorized access to user accounts or administrative systems. Compromised accounts can lead to data theft, financial loss, and unauthorized system modifications.

Vulnerability path:
http://testphp.vulnweb.com/login.php

Vulnerability parameter:
username/password

Steps to Reproduce:
➢ Navigate to the login page.
➢ Attempt multiple login attempts using different passwords.
➢ Observe that the system does not lock the account after repeated failures.

Screenshot:



Recommendation:
• Implement account lockout policies after multiple failed login attempts.
• Enforce strong password policies.
• Use multi-factor authentication for sensitive accounts.

Vulnerability Report 8

Vulnerability name: Unsigned or Unverified Software Updates
CWE: 494
OWASP category: A08:2021 – Software and Data Integrity Failures

Description:
This vulnerability occurs when applications download or install updates without verifying the authenticity or integrity of the update files. Attackers may intercept or modify update packages and distribute malicious code.

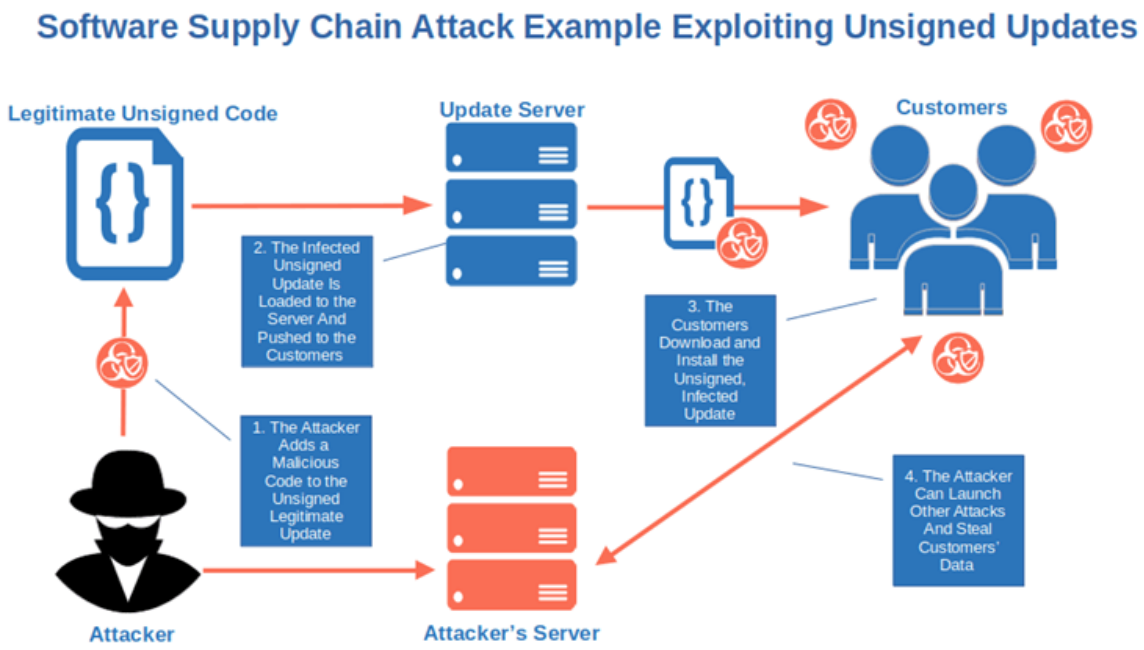
Business Impact:
Attackers may distribute malicious software disguised as legitimate updates. This can lead to system compromise, malware installation, and unauthorized access to internal networks.

Vulnerability path:
http://localhost/update-service

Vulnerability parameter:
Update file URL

Steps to Reproduce:
➢ Attempt to upload or replace the update file with a modified version.
➢ Observe that the application installs the update without verifying its digital signature.

Screenshot:



Recommendation:
• Use digital signatures to verify software updates.
• Validate file integrity using hashing algorithms.
• Restrict update sources to trusted repositories.

Vulnerability Report 9

Vulnerability name: Insufficient Logging and Monitoring
CWE: 778
OWASP category: A09:2021 – Security Logging and Monitoring Failures

Description:
This vulnerability occurs when applications fail to properly log security events such as login attempts, access violations, or suspicious activities. Without proper logging, it becomes difficult to detect attacks or investigate incidents.

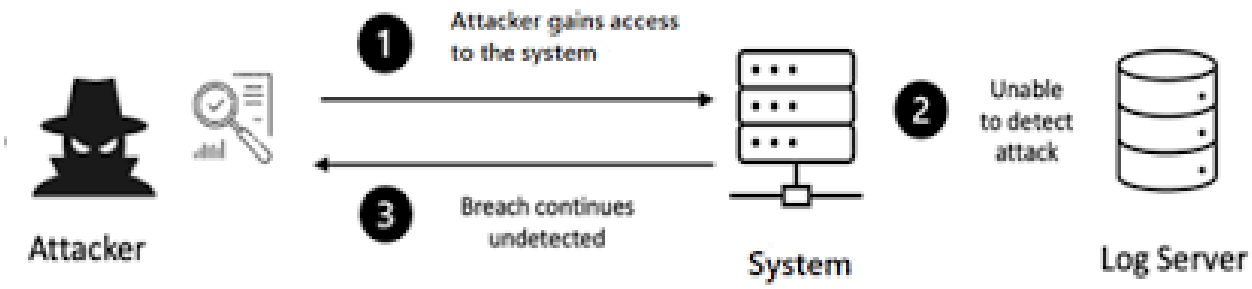
Business Impact:
Attackers may perform malicious activities without detection. Organizations may not be able to identify security breaches quickly, resulting in prolonged exposure and greater damage.

Vulnerability path:
http://localhost/login

Vulnerability parameter:
Authentication events

Steps to Reproduce:
➢ Attempt multiple failed login attempts.
➢ Access system logs.
➢ Observe that the login attempts are not recorded.

Screenshot:



Recommendation:
• Implement centralized logging systems.
• Log authentication attempts and suspicious activities.
• Configure security monitoring alerts.

Vulnerability Report 10

Vulnerability name: Server-Side Request Forgery (SSRF)
CWE: 918
OWASP category: A10:2021 – Server-Side Request Forgery

Description:
SSRF vulnerabilities occur when an application fetches remote resources based on user-supplied input without proper validation. Attackers can manipulate the request to access internal resources or services that are not intended to be publicly accessible.

Business Impact:
Attackers may access internal systems, cloud metadata services, or administrative interfaces. This can result in sensitive data exposure or further compromise of internal infrastructure.

Vulnerability path:
https://portswigger-labs.net/product/stock

Vulnerability parameter:
stockApi

Example Payload:

http://localhost/admin

Steps to Reproduce:
➢ Intercept the stock API request using a proxy tool.
➢ Modify the URL parameter to access an internal resource.
➢ Send the request to the server.
➢ The server returns internal system data.

Screenshot:

Recommendation:
• Validate and restrict external resource requests.
• Implement allowlists for approved URLs.
• Block access to internal IP ranges.

Vulnerability Report 11

Vulnerability name: Lack of Isolation Between Services
CWE: 653
OWASP category: Security Architecture – Improper Network Segmentation

Description:
Improper network segmentation occurs when internal services are not properly isolated from each other. If one service becomes compromised, attackers may move laterally across the network to access other internal resources. This vulnerability often occurs when systems are deployed without segmentation policies or micro-segmentation controls.

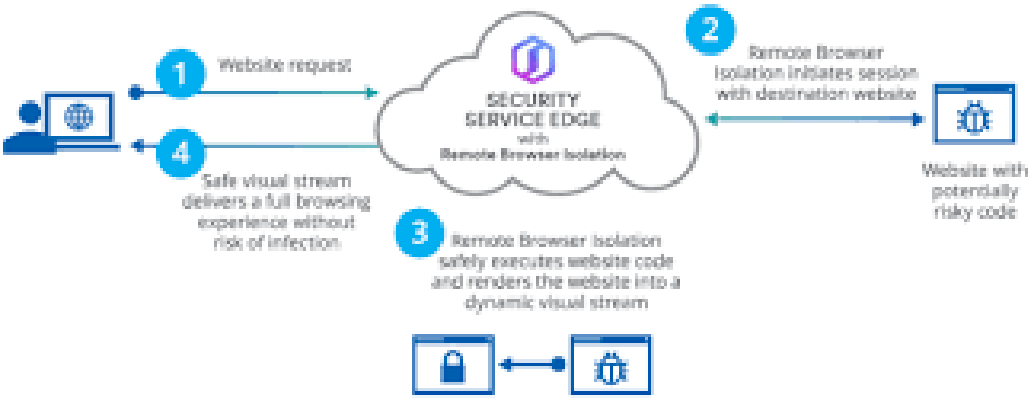
Business Impact:
Attackers who gain access to one application or server may exploit the lack of isolation to move laterally within the network. This could lead to compromise of sensitive systems such as databases, authentication services, or administrative platforms.

Vulnerability path:
http://internal-server/admin

Vulnerability parameter:
Internal service access through exposed network port

Steps to Reproduce:
➢ Gain access to one internal service within the network.
➢ Attempt to connect to another internal service using its internal IP address.
➢ Observe that the connection is allowed without restriction.

Screenshot Process:



Recommendation:
• Implement network segmentation policies.
• Apply Zero Trust network access controls.
• Restrict communication between services using firewall rules.

Vulnerability Report 12

Vulnerability name: Weak TLS Configuration
CWE: 319
OWASP category: A02:2021 – Cryptographic Failures

Description:
Weak TLS configuration occurs when applications use outdated encryption protocols or insecure cipher suites. These weak configurations may allow attackers to intercept and decrypt sensitive communications between users and the application.

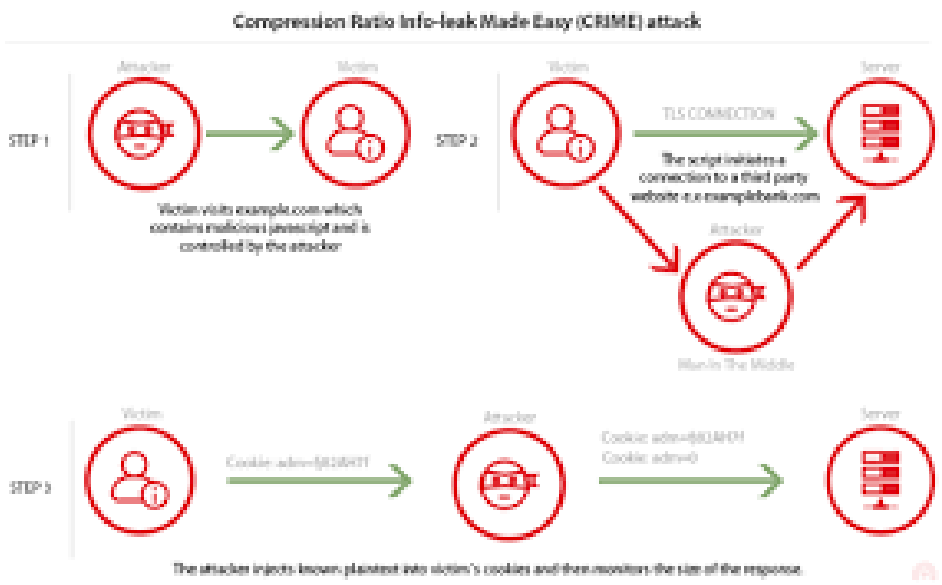
Business Impact:
Sensitive information such as login credentials, personal data, or financial information may be intercepted by attackers through man-in-the-middle attacks.

Vulnerability path:
https://testssl.example.com

Vulnerability parameter:
TLS protocol version

Steps to Reproduce:
➢ Scan the target server using an SSL/TLS scanning tool.
➢ Identify outdated protocols such as TLS 1.0 or weak cipher suites.
➢ Verify that the server supports insecure encryption methods.

Screenshot:



Recommendation:
• Disable outdated TLS protocols.
• Enable TLS 1.2 or TLS 1.3.
• Use strong encryption cipher suites.

Vulnerability Report 13

Vulnerability name: Insecure Identity Federation / SSO Issues
CWE: 306
OWASP category: A07:2021 – Identification and Authentication Failures

Description:
Improper identity management occurs when authentication mechanisms such as Single Sign-On (SSO) or identity federation are incorrectly implemented. Attackers may exploit these flaws to bypass authentication or impersonate legitimate users.

Business Impact:
Attackers may gain unauthorized access to sensitive systems or user accounts. If administrative accounts are compromised, attackers may gain complete control over the system.

Vulnerability path:
https://example.com/sso-login

Vulnerability parameter:
SSO authentication token

Steps to Reproduce:
➢ Attempt authentication through the SSO login system.
➢ Intercept the authentication request.
➢ Modify the authentication token or parameters.
➢ Forward the request and observe if authentication bypass occurs.

Screenshot:

Recommendation:
• Validate authentication tokens securely.
• Implement strong identity verification mechanisms.
• Use secure token validation and expiration policies.

Vulnerability Report 14

Vulnerability name: Misconfigured Cloud Storage or Access Policies
CWE: 284
OWASP category: Cloud Security Misconfiguration

Description:
Cloud misconfiguration occurs when cloud resources such as storage buckets or databases are configured with overly permissive access controls. This may allow unauthorized users to access sensitive data stored in the cloud.

Business Impact:
Sensitive corporate data, user information, or confidential documents may be publicly accessible. This can result in data breaches and regulatory compliance violations.

Vulnerability path:
https://public-bucket-example.s3.amazonaws.com

Vulnerability parameter:
Public access configuration

Steps to Reproduce:
➢ Attempt to access the cloud storage URL through a browser.
➢ Observe whether files stored in the bucket are publicly accessible.
➢ Verify if downloading files requires authentication.

Screenshot:

Recommendation:
• Disable public access for sensitive storage buckets.
• Apply strict identity and access management policies.
• Monitor cloud configurations regularly.

Vulnerability Report 15

Vulnerability name: Unverified Endpoint or Device Posture
CWE: 295
OWASP category: Device Security Validation Failure

Description:
Insufficient device trust validation occurs when applications allow access from unmanaged or unverified devices without verifying their security posture. Attackers can exploit this weakness by connecting from compromised devices.

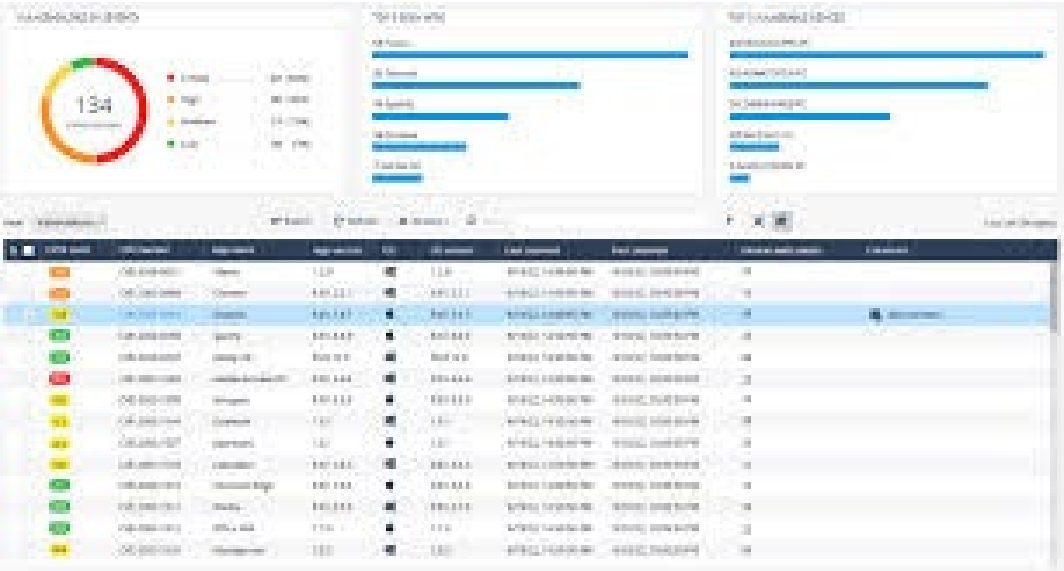
Business Impact:
Unauthorized devices may gain access to internal systems or sensitive applications. Compromised endpoints can be used as entry points for malware or data exfiltration.

Vulnerability path:
https://internal-application.example.com

Vulnerability parameter:
Device authentication verification

Steps to Reproduce:
➢ Attempt to access the application from an unmanaged or unknown device.
➢ Observe whether the application grants access without verifying device security status.

Screenshot:



Recommendation:
• Implement device posture validation.
• Allow access only from trusted and compliant devices.
• Enforce Zero Trust security policies.

STAGE-2

Nessus

Overview:

Nessus is a popular vulnerability assessment tool developed by Tenable that is used to identify security weaknesses in computer systems, networks, servers, and web applications. It helps organizations detect vulnerabilities such as outdated software, missing security patches, weak configurations, open ports, and potential malware that attackers could exploit. Nessus works by scanning the target systems and comparing the results with its continuously updated database of known vulnerabilities and security issues.

The tool uses a large collection of plugins to perform different types of security checks and vulnerability tests. These plugins allow Nessus to detect thousands of known vulnerabilities across various operating systems, databases, applications, and network devices. After completing a scan, Nessus generates detailed reports that categorize vulnerabilities based on their severity levels such as critical, high, medium, and low. The reports also include a description of each vulnerability, its potential impact, and recommended solutions or remediation steps to fix the problem.

Nessus supports multiple scanning techniques including network scanning, configuration auditing, malware detection, and web application vulnerability scanning. It also allows users to schedule scans, customize scanning policies, and monitor system security regularly. Because of its high accuracy, user-friendly interface, and regular updates, Nessus is widely used by cybersecurity professionals, penetration testers, and organizations to strengthen their security posture and protect systems from potential cyber threats

use cases of Nessus:

- Vulnerability Assessment
  Nessus is widely used to identify security vulnerabilities in networks, servers, and systems. It scans devices to detect weaknesses such as outdated software, unpatched systems, and insecure configurations that attackers could exploit.
- Network Security Monitoring
  Organizations use Nessus to monitor their network security by scanning routers, switches, firewalls, and other network devices. This helps security teams detect open ports, weak protocols, and other network-related vulnerabilities.
- Compliance Auditing
  Nessus helps organizations meet security compliance requirements such as PCI-DSS, HIPAA, and ISO standards. It checks systems against security policies and compliance guidelines to ensure that organizations follow required security practices.
- Patch Management Support
  Nessus identifies missing patches and outdated software versions on systems. This allows administrators to update and patch their systems quickly to reduce security risks.
- Web Application Security Testing
  Security professionals use Nessus to scan web applications for vulnerabilities like SQL injection, cross-site scripting (XSS), and other common web security issues.
- Configuration Auditing
  Nessus checks system configurations to identify weak settings such as default passwords, unnecessary services, or improper security configurations that could lead to security breaches.
- Penetration Testing Support
  Ethical hackers and penetration testers use Nessus as a reconnaissance tool to gather vulnerability information before performing deeper security testing.
- Risk Management
  Nessus provides vulnerability severity ratings and detailed reports, helping organizations prioritize security issues and fix the most critical vulnerabilities first.

Features of Nessus

1. **Comprehensive Vulnerability Detection**
   Nessus can detect thousands of vulnerabilities in operating systems, network devices, databases, and web applications. It uses a large and frequently updated plugin database to identify security weaknesses.
2. **High-Speed Asset Discovery**
   The tool quickly scans networks to discover active hosts, open ports, and running services. This helps security teams understand what devices are connected to the network.
3. **Configuration Auditing**
   Nessus checks system configurations and security settings to ensure they follow

recommended security standards. It can detect weak configurations such as default credentials or insecure protocols.

4. **Malware Detection**
   Nessus helps identify systems that may be infected with malware or communicating with suspicious or malicious hosts.
5. **Web Application Scanning**
   It scans web applications for common vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and other security flaws.
6. **Detailed Vulnerability Reports**
   After scanning, Nessus generates detailed reports that include vulnerability descriptions, severity levels (Critical, High, Medium, Low), and recommended solutions.
7. **Customizable Scan Policies**
   Users can create customized scanning policies depending on the network environment and security requirements.
8. **Regular Plugin Updates**
   Nessus receives regular updates from Tenable to detect newly discovered vulnerabilities.

How to Use Nessus

1. **Install Nessus**
   Download and install Nessus from the official website of Tenable. It is available for Windows, Linux, and macOS systems.
2. **Activate the Tool**
   After installation, activate Nessus using an activation code provided during registration.
3. **Access the Web Interface**
   Nessus runs through a web-based interface. Open a browser and access it using: https://localhost:8834
4. **Create a New Scan**
   Click **New Scan** and choose a scan template such as **Basic Network Scan**.
5. **Configure the Scan**
   Enter the **target IP address or domain name** of the system you want to scan and configure scan settings if required.
6. **Run the Scan**
   Start the scan. Nessus will analyze the target system and check for vulnerabilities.
7. **Review the Results**
   After the scan is completed, Nessus displays the vulnerabilities found along with their severity levels.
8. **Generate Reports**
   Export the scan results as reports (PDF, HTML, or CSV) for documentation and further security analysis.

Target website: http://testphp.vulnweb.com/
Target ip address: 172.217.26.20

List of vulnerability

S.No	Vulnerability Name	Severity	Plugin Id
1	HTTP (Multiple Issues)	Info	10107
2	Web Server (Multiple Issues)	Info	11239
3	Service Detection	Info	22964
4	Nessus SYN Scanner	Info	11219
5	Host Fully Qualified Domain Name (FQDN) Resolution	Info	12053
6	Nessus Scan Information	Info	19506
7	OS Fingerprints Detected	Info	209654
8	Traceroute Information	Info	10287

Reports:

1. Vulnerability Name: HTTP (Multiple Issues)

Severity: Info

Plugin: 10107

Port: 80

Description: This plugin reports general information about the HTTP service running on the target system. It identifies that the system is using an HTTP web server and may include multiple informational checks related to HTTP headers, responses, or server configuration. The result does not indicate a direct vulnerability but provides details about the web service detected during the scan.

Solution: Review the web server configuration and ensure that it is properly configured with secure settings. Keep the web server software updated and disable unnecessary services or modules.

Business Impact: Attackers may gather information about the web server configuration which could be used for reconnaissance and planning future attacks.

2. Vulnerability Name: Web Server (Multiple Issues)

Severity: Info

Plugin: 11239

Port: 80

Description: This plugin provides general information about the detected web server. It may identify the server type, version, or configuration details that help administrators understand the web services running on the host. It is mainly informational and does not indicate a confirmed vulnerability.

Solution: Regularly update the web server software and review its configuration to remove unnecessary information disclosure and unused components.

Business Impact: Information about the web server can help attackers identify possible weaknesses and target specific exploits related to the server technology.

3. Vulnerability Name: Service Detection

Severity: Info

Plugin: 22964

Port: Multiple Ports

Description: This plugin detects and identifies services running on various open ports of the target system. It helps determine which network services are available and how they respond to network requests.

Solution: Review all detected services and disable any unnecessary or unused services. Ensure that only required services are exposed to the network.

Business Impact: Unnecessary services increase the attack surface and may provide attackers with additional entry points into the system.

4. Vulnerability Name: Nessus SYN Scanner

Severity: Info

Plugin: 11219

Port: Multiple Ports

Description: This plugin indicates that the scan used the SYN scanning technique to identify open ports on the target host. SYN scanning helps detect active services without completing full TCP connections.

Solution: Network administrators should review open ports and close any ports that are not required for business operations. Implement firewall rules to restrict unnecessary network access.

Business Impact: Open ports may expose services that attackers can attempt to exploit if not properly secured.

5. Vulnerability Name: Host Fully Qualified Domain Name (FQDN) Resolution
Severity: Info
Plugin: 12053
Port: N/A
Description: This plugin resolves and reports the Fully Qualified Domain Name (FQDN) of the scanned host. It helps identify the domain name associated with the system's IP address.
Solution: Ensure that DNS configurations are properly maintained and that unnecessary DNS information is not publicly exposed.
Business Impact: Revealing domain information may assist attackers in gathering details about the organization's network structure.

6. Vulnerability Name: Nessus Scan Information
Severity: Info
Plugin: 19506
Port: N/A
Description: This plugin provides information about the scan itself, such as the scanning engine, plugin set, and scan configuration. It is used for reporting and auditing purposes.
Solution: No direct remediation is required since this plugin only provides scan-related information.
Business Impact: There is no direct security impact, but the information is useful for documentation and compliance purposes.

7. Vulnerability Name: OS Fingerprints Detected
Severity: Info
Plugin: 209654
Port: N/A
Description: This plugin attempts to identify the operating system of the target host by analyzing network responses. The detected OS fingerprint helps determine the system type and version.
Solution: Keep the operating system updated with the latest security patches and minimize information leakage that could reveal system details.
Business Impact: If attackers identify the operating system, they may attempt exploits that specifically target that platform.

8. Vulnerability Name: Traceroute Information

Severity: Info

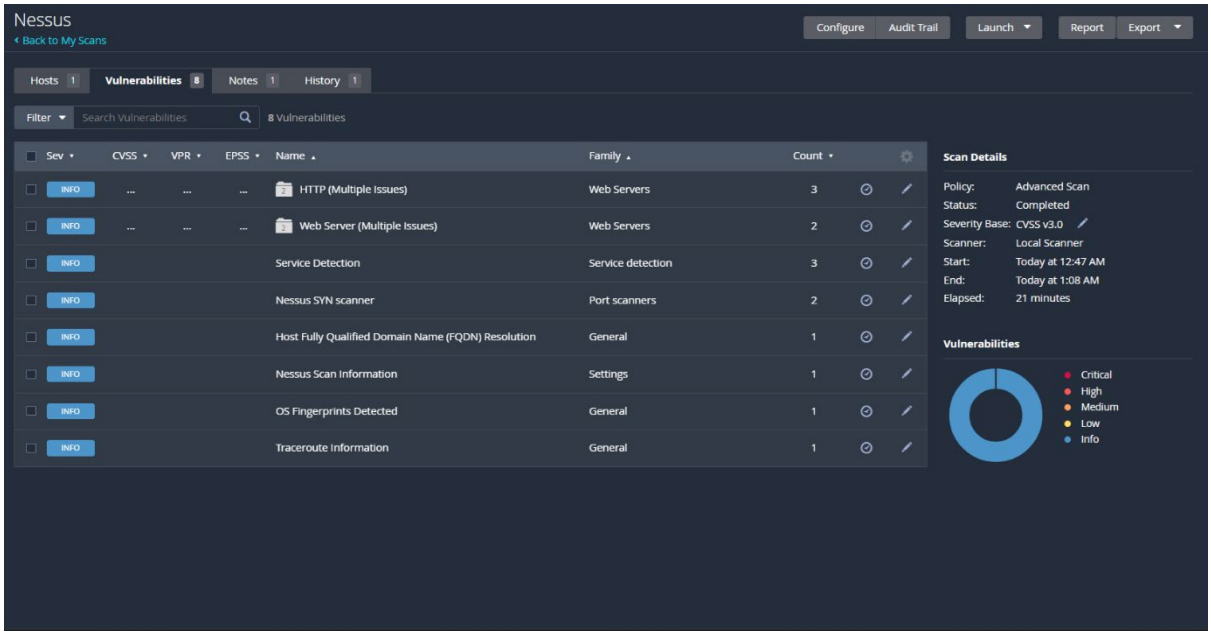
Plugin: 10287

Port: N/A

Description: This plugin performs a traceroute to identify the network path between the scanning system and the target host. It provides details about intermediate network devices and routing paths.

Solution: Restrict unnecessary ICMP responses or traceroute visibility where possible using firewall or network configurations.

Business Impact: Network path information may help attackers understand the network infrastructure and identify potential targets for further attacks.



STAGE-3

REPORT

Title: Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

User Identity Verification:

User identity verification is a fundamental aspect of context-aware access control, as it ensures that the person requesting access is genuinely who they claim to be. This process involves authenticating users through credentials such as usernames, passwords, biometric authentication, or multi-factor authentication. By accurately verifying the identity of users, organizations can ensure that only authorized individuals gain access to sensitive systems and information. Identity verification also helps in assigning appropriate access permissions based on the user's role, responsibilities, and level of trust within the organization.

Device Posture and Security Status:

Device posture refers to the security condition of the device being used to access a system or application. In context-aware access control, the system evaluates whether the device meets the required security standards before granting access. This includes checking if the device has updated antivirus software, the latest operating system patches, proper security configurations, and whether it is managed by the organization. Devices that do not meet these requirements may be restricted, denied access, or allowed only limited access to protect the organization's systems from potential threats.

Location and Application Risk Evaluation:

Another important subtopic in context-aware access control is evaluating the location of the access request and the risk level of the application being accessed. The system analyzes whether the request is coming from a trusted network, corporate office, or an unfamiliar geographic location. At the same time, it considers the sensitivity of the application or data involved. High-risk applications such as financial systems or confidential databases require stricter access policies compared to low-risk applications. By combining location data with application risk levels, organizations can make smarter access decisions that enhance security while still allowing legitimate users to perform their tasks effectively.

Conclusion:

Stage1:

The first stage of the project focused on understanding the concept of context-aware access control and its importance in modern cybersecurity environments. Traditional access control mechanisms rely mainly on user authentication, but they are not sufficient to protect systems from advanced cyber threats. In this stage, the project explored how access decisions can be improved by evaluating contextual factors such as user identity, device posture, location, and application risk. By analyzing these factors, organizations can implement dynamic security policies that provide stronger protection against unauthorized access. The study also demonstrated how context-aware security models support modern security frameworks such as Zero Trust Architecture. Overall, this stage provided a strong conceptual foundation for implementing intelligent and adaptive access control mechanisms.

Stage2:

In the future, context-aware access control systems can be enhanced by integrating advanced technologies such as artificial intelligence, machine learning, and behavioral analytics. These technologies can automatically analyze user behavior patterns and detect suspicious activities in real time. Organizations can also integrate context-aware access control with cloud security platforms, identity and access management systems, and mobile device management solutions to provide stronger security across distributed environments. Additionally, implementing automated risk-based authentication and continuous monitoring can further improve the efficiency and reliability of access control systems.

Stage3:

The third stage of the project focused on applying context-aware access control concepts through detailed analysis of security mechanisms such as user identity verification, device posture validation, and location-based risk evaluation. This stage demonstrated how modern security systems use contextual data to dynamically control user access to applications and resources. By analyzing these different contextual parameters, organizations can reduce the risk of unauthorized access, data breaches, and insider threats. The study highlighted the importance of integrating strong authentication methods, secure device validation, and risk-based access policies to protect sensitive systems and data.

Future Scope:

Stage1:

In the future, context-aware access control systems can be enhanced by integrating advanced technologies such as artificial intelligence, machine learning, and behavioral analytics. These technologies can automatically analyze user behavior patterns and detect suspicious activities in real time. Organizations can also integrate context-aware access control with cloud security platforms, identity and access management systems, and mobile device management solutions to provide stronger security across distributed environments. Additionally, implementing automated risk-based authentication and continuous monitoring can further improve the efficiency and reliability of access control systems.

Stage2:

Future work can include performing deeper vulnerability assessments and penetration testing on complex environments such as enterprise networks and cloud infrastructures. Advanced scanning tools can be integrated with automated patch management systems to quickly remediate detected vulnerabilities. Organizations can also implement continuous security monitoring and vulnerability management frameworks to improve overall system security. In addition, combining vulnerability scanners with threat intelligence platforms can help detect emerging cyber threats and improve incident response capabilities.

Stage-3:

Future improvements can include implementing context-aware access control solutions within real enterprise environments using identity and access management platforms. Organizations can also integrate behavioral analytics, biometric authentication, and real-time threat detection systems to enhance access control mechanisms. The use of cloud-based identity services and adaptive authentication systems will further improve security and usability. In addition, future research can focus on developing automated policy engines that dynamically adjust access permissions based on risk analysis and environmental context.

Topics Explored:

Context-Aware Access Control
Zero Trust Security Model
User Identity Verification
Device Posture Assessment
Location-Based Access Control
Application Risk Evaluation

Tools Explored

Conceptual research tools and cybersecurity frameworks

Online vulnerability research platforms

Security documentation resources

Nessus

Web browser interface for security scanning

Vulnerability reporting tools